

Apologies if this exceeds the word limit; the details are necessary to convey the story accurately and would lose essential context if over-compressed.

1) Rationale, evidence, success, decisions, and outcome

Rationale

On Talabat's Home of Food, user attention was overly concentrated at the very top of the list:

- 71% of clicks happened in the first 10 results;
- Only 18% of sessions scrolled past row 6;
- and 72% of orders came from ~8% of chains.
- Conversion dropped steeply by rank (rank-1 3.3% vs. rank-12 0.9%).

Our hypothesis was that a relevance-first retrieval model would better match supply to intent, increase orders and GMV, and improve discovery—all without harming delivery or quality guardrails.

Evidence it was worth pursuing (DA analysis; 8 weeks, 6 markets, ~2.4M sessions/day)

Exit concentration. On generic intents ("burger", "pizza"), exits after row-1 were +24% higher than on brand queries—users weren't finding a good fit quickly.

Exposure imbalance. Head chains took 58% of impressions/64% of orders; long-tail chains got 8% of impressions but converted +22% better when they were actually shown (given-exposure conversion) → under-exposed relevant supply.

Daypart misalignment. Only 37% of top-10 impressions in Breakfast (07:00–11:00) went to breakfast-leaning vendors, even though matched vendors converted 2.9% vs. 1.7% mismatched (+1.2pp). Similar gaps at Dinner (3.8% vs. 2.6%), Late-night (2.2% vs. 1.3%), Weekend brunch (3.1% vs. 1.9%). A counterfactual reallocation of 15% of misaligned impressions projected +2.1% orders/session and +1.5% GMV/session.

How success was defined

North-Star: Orders per session up, GMV per session up.

Primary metrics: CTR at Top-10 up (as a signal that people see better options), orders per ordering customer up or flat-positive, and AOV neutral to up.

Secondary metrics (discovery and journey): distinct chains ordered per 10k sessions up, new-to-user order rate up, time to first click down

Business guardrails: unique active customers flat to up; ordering customers flat to up; sessions may go down if people decide faster, but orders and orders per customer must go up; contribution margin per order flat to up.

Operational guardrails: cancellations $\leq +0.2pp$, ETA drift $\leq +0.5$ min, preparation-time P95 no worse, menu/stock errors no worse.

What we built (two-tower retrieval in TensorFlow Recommenders, see architecture in appendix)

We created a custom Recall@10 defined as “was the chain actually ordered in Top-10 predictions?” We stuck to that metric in all our offline evaluation and used it for different segments (new chains = discovery, popular chains, expensive/cheap chains etc that we used in the form of tiers insights)

User tower signals:

- account_id
- recent vendors/clicks/searches (tokenized + shared embeddings + light self-attention)
- frequent chains ordered
- order_hour/order_weekday
- delivery area
- geohash
- order source.
- is subscribed to premium options (monthly paid)

Vendor tower signals:

- chain_id;
- cuisine text;
- name text (shared keyword embedding for semantic generalization);
- daily orders
- average number of order per hour of day
- avg GMV
- rating;
- area coverage (frequent service areas).

Pretrained vendor embeddings (behavioral, like item2vec), from our internal ESE team (engine for similar entities).

ESE team learned restaurant vectors from co-visitation/co-ordering patterns. The vendor tower looks up each restaurant's pretrained vector (kept frozen for stability). The user tower aggregates vectors of the customer's recent restaurants (time-decayed). This anchors both towers in the same semantic space from day one, improving overall relevance, speeding convergence, and strengthening cold-start.

Complex decision we made (addressing popularity bias in training)

Our first A/B vs. control (default + Elastic) lifted CTR (+9%, p-value=0.05) and orders (+5.4%, p-value=0.05) but reduced discovery (distinct chains ordered -3%, p-value=0.05).

Root cause: popularity bias—popular chains appear far more often in training as candidates, so their gradients dominate.

How training data is built (Important):

Each example comes from one session. The **positive** is the chain the user ordered; **negatives** are other chains that were viable (shown/eligible) but not ordered in the same context. The model learns to score the positive above the negatives. Without correction, frequent head chains overwhelm the loss. (add appendix for the loss)

Chosen fix (training-time debiasing):

We estimated each chain's candidate probability (how often it appears among candidates across mixed positive/negative batches) and fed this into the loss as a sampling probability with chain IDs (inverse-propensity style). We also used positive sample weights and a mild temperature to avoid over-confident peaking. Net effect: gradients no longer collapse onto head chains; long-tail quality signals get learned. (see appendix)

What was the final outcome (before correcting bias, see later the outcome when applying the correction)

Offline evaluation (UAE/Egypt focus; 3.5-week train / 1-week test):

- Recall@10 +5.7 points overall (vs baseline to beat **estimated per market**, inspired from the literature related to good recall values for a ranking model);
- Long-tail Recall@10 +8–10%; better calibration.

We achieved this via large-scale hyperparameter search (Vertex AI) over negative-sampling ratio (ns_ratio), batch size, temperature, and light label smoothing, enabled by faster TFRecord datasets (we generated multiple variants to iterate quickly).

Online A/B (21 days, 6 markets, Home of Food vs. control): North-Star

- Orders/session: +5.4% (p-value=0.05)
- GMV/session: +7.9% (p-value=0.03)

Primary

- CTR @ Top-10: +9.1% (p-value=0.05)
- Orders per ordering customer: +1.6% (p-value=0.05)
- AOV: +0.1% (neutral; ns) (p-value=0.05)

Secondary (discovery & journey)

- Distinct chains ordered / 10k sessions: -3.2% ← discovery down (p-value=0.05)
- New-to-user order rate: -0.8pp ← discovery down (p-value=0.05)
- Time to first click: -12% (faster decision) (p-value=0.05)

Business guardrails

- Unique active customers: +0.3% (ns)
- Ordering customers: +1.0%
- Sessions: -5.5% (expected—fewer back-and-forth searches)

Bottom line: We hit the North-Star and primary goals and stayed within guardrails, but discovery dropped (-3.2% distinct chains; -0.8pp new-to-user). This confirmed popularity bias and motivated the training-time debiasing fix in the next iteration.

2) One decision in depth: correcting popularity bias

Problem we observed in the first A/B.

We saw strong gains in CTR and orders, but discovery fell (-3% distinct chains ordered). Popular chains were shown and clicked more, and because they also appeared more frequently in training, the model's gradients focused on them even more. **This is the classic popularity bias feedback loop.**

How the training data works (Reminder)

Every training example comes from one session on Home of Food.

-The positive sample is the chain the customer actually ordered.

-The negative samples are the other chains that were available (shown/eligible) in that same context but not ordered.

The model learns to score the positive higher than the negatives. If a popular chain shows up in 40% of training batches while a niche chain shows up in 2%, the popular chain gets roughly 20× the gradient updates unless we correct for it. (see appendix)

Below are the realistic options we considered for a food-delivery 2-tower system, with clear pros/cons, how we would evaluate each, and why we picked the final path.

A) Serve-time diversification (changes after the model scores)

Heuristic coverage rules.

Example rules are “always show at least one breakfast place in the morning” or “don’t show too many restaurants from the same chain group.”

- **Pros:** These rules are quick to add. They give us immediate control without retraining the model.
- **Cons:** They can break in different cities or time slots. They add code that must be maintained at serving time. They may fight the model’s scores and can be hard to tune fairly for small vendors vs. big brands.
- **How we would measure:** We would look at CTR, orders per session, fairness across vendor size (by deciles), how well daypart matches improved, and any extra latency.

Slate-aware re-ranking.

This means reordering the top results to balance “relevance” with “variety” using a more formal method.

- **Pros:** It is a principled way to make the final Top-K list both relevant and diverse.
- **Cons:** It adds complexity and some latency. It needs high-quality labels like cuisine, area, and brand group. It is hard to scale across all markets.
- **How we would measure:** Same as above, plus specific “slate diversity” metrics and stability under high traffic.
- **Why we did not pick A as the main fix:**
These methods treat the final list rather than the root cause (how the model learns from biased training data). They also increase serving complexity across many markets.

B) Exploration strategies (learning online while we show results)

Contextual bandits (e.g., ϵ -greedy, Thompson sampling).

This means we purposely try some uncertain restaurants to learn if they perform well.

- **Pros:** It is a solid way to discover good long-tail options over time.
- **Cons:** It carries product and operations risk (fees, ETA, SLAs). It needs strict guardrails. It usually takes longer to learn. Stakeholders may not like deliberate exploration during peak times.
- **Why we did not pick B first:**
This is the right long-term direction, but it was too heavy for our immediate goals and risk level. It fits better as phase two once the basics are stable.

C) Training-time fixes (the path we chose)

Debiased sampling / inverse-propensity weighting inside the loss.

What this means for our 2-tower model:

- We build training pairs from each session: the positive is the restaurant the user ordered; the negatives are other restaurants that were shown or eligible but not ordered.
- We estimate, for each restaurant, how often it appears as a candidate in training (its exposure probability).
- We pass that probability (with the restaurant's ID) into the training loss. This reduces the weight of over-exposed big brands and increases the weight of under-exposed long-tail restaurants. (see appendix)
- We also add sample weights for positives and use a small temperature to prevent the model from becoming over-confident too quickly.

- **Pros:** This fixes the root cause during learning. It does not add new serving logic. It works across dayparts and markets. It keeps latency low.

- **Cons:** We must estimate exposure carefully. If we push too hard, short-term CTR can drop. It requires strong offline evaluation to tune correctly.

Hard / semi-hard negative mining.

This adds negatives that are “close competitors,” such as restaurants often shown together or with similar menus.

- **Pros:** It teaches the model to separate similar options (e.g., two burger places at dinner).

- **Cons:** If we mine from biased logs, we can reintroduce bias. We should mix these with uniform or exposure-aware negatives.

- **How we used it:** As a supplement, not the main fix.

Loss shaping (focal-style weighting, label smoothing).

- **Pros:** These techniques make training more stable and reduce over-confidence.

- **Cons:** They are indirect and need careful tuning.

- **How we used it:** We applied a light temperature and light label smoothing to stabilize training.

Why we chose C:

It corrects exposure bias during training, keeps the serving system simple, and fits well with our two-tower retrieval approach (fast approximate nearest-neighbor search and a clean architecture).

How we de-risked and proved it works.

Massive offline evaluation: we defined Recall@10 as “is the actually ordered chain in our predicted Top-10?”, and we sliced results by market, cuisine, geography, chain number of orders (tiers), chain popularity (tiers), average basket price (tiers) automated reports to detect low performance in long tail chains (last tiers)

Faster datasets for iteration: we wrote training data as TFRecords and generated

multiple variants with different negative-sampling ratios (ns_ratio) and batch sizes, which dramatically reduced data-prep time and let us run many experiments in parallel.

Large-scale hyperparameter search (Vertex AI): we searched ns_ratio, batch size, temperature, and light label smoothing, selecting the best configuration by global Recall@K and Long-tail@K while keeping latency budgets.

Cross-market robustness: primary focus on UAE and Egypt (biggest markets), with monitoring across Kuwait, Qatar, Bahrain, Iraq, Jordan, Oman to ensure the approach generalized well.

A/B Tests: We validate all our assumptions by running experimentations, after we make sure we have solid offline results.

Outcome of the decision

Offline (UAE/Egypt; 3.5-week train / 1-week test).

Recall@10: +6.9 pts overall

Long-tail Recall@10: +11%

Online A/B (21 days, 6 markets, Home of Food vs. control).

North-Star

Orders/session: +5.0% (p-value=0.05)

GMV/session: +7.1% (p-value=0.03)

Primary

CTR @ Top-10: +8.4% (p-value=0.05)

Orders per ordering customer: +1.9% (p-value=0.05)

AOV: +0.3% (neutral→slightly up)

Secondary (discovery & journey)

Distinct chains ordered / 10k sessions: +9%  (p-value=0.05)

New-to-user order rate: +2.4pp  (p-value=0.05)

Time to first click: -11% (faster decision)

Business guardrails

Unique active customers: +0.5% (ns) (p-value=0.05)

Ordering customers: +1.3% (p-value=0.05)

Sessions: -5.2% (expected—less back-and-forth) while Orders: +6.0% and Orders/customer: +2.1%

Business interpretation: although sessions fell 5–6%, ordering customers rose and orders/sessions rose—a sign of fewer steps to decision rather than reduced demand. Operational guardrails stayed within limits.

This decision attacked the root cause of popularity bias inside training, avoided serve-time complexity, and delivered a healthier marketplace: better matches quickly, more diverse vendors getting fair exposure, and measurable gains on core KPIs.

3) How I approached delivery

We kept the team small and clear:

- **A Data Engineer** moved data between BigQuery and Postgres and ran the nightly jobs for restaurant (chain) vectors.
- **A Software Engineer** owned the production service on Kubernetes: taking a user request, doing fast similarity search, and applying business rules (availability, de-duplication, “busy” push-down) = endpoint used by the vendor team.
- **I owned** the model and the production-ready package: training, making real-time inference fast, keeping training and serving perfectly consistent, and validating end-to-end results.

How we scoped the work

We split the system in 3 so we could move in parallel:

Chain side (batch, offline). Use the trained model and compute a vector for every restaurant (night batch job). Land those vectors in BigQuery and then Postgres for the service to use. (I owned with a Data engineer)

User side (real-time). Given a live user request, create the user vector in ≤ 20 ms, find the closest restaurant vectors, and return the ranked list (owned by software engineer + my contribution)

One deployable artifact. Package the model as a single MLflow artifact and Docker image so the Software Engineer could deploy without re-implementing anything. The artifact recreates the exact training architecture and loads the same vocabularies, list of chains, area ids, cuisines etc and weights (I owned)

What we prioritised early

Get chain embeddings into Postgres and prove read speed.

We computed chain vectors first and pushed them to Postgres. In UAE we had ~13k chains; we verified the service could load them in memory and run cosine similarity quickly (on 300 chains, way above the maximum we could get from vendor API). With a simulated user vector, vector load + Top-K ranking took about ~10 ms, which gave us confidence the serving path would be fast enough.

Speed first. I made the user-tower forward pass fast enough for production (target ≤ 20 ms). I pre-loaded heavy layers at start-up, cached text processors, and removed unnecessary steps.

Same features everywhere. I created one feature module that both training and serving use, including the same “fill empty values” rules (e.g., no previous orders → use the default token). This avoided silent differences.

A clean contract with Engineering. We agreed on cosine similarity over L2-normalised vectors, one tie-break rule for ties, and that the service will only rank restaurants that are actually available.

The biggest problems we hit — and how we fixed them

1. Latency was too high at first.

The user tower (inference) ran slower than our 20 ms budget.

Fix: We pre-computed and loaded the pretrained restaurant embeddings at start-up. We built and warmed user towers at startup: ran tiny dummy inputs to build the TF graph and loaded weights before any traffic. Wrapped the forward call in `tf.function` to compile it, and warmed with multiple input signatures to avoid re-tracing later.

2. Training vs. serving didn't match (feature drift).

The defaults for missing values changed in training but not in production, so outputs diverged.

Fix: We made a single source of truth for feature logic and shipped it inside the MLflow artifact. Training and serving now use the exact same code and defaults.

3. ID types were wrong in production.

The service converted `account_id` to integers, but we trained on strings. That broke lookups and rankings.

Fix: We enforced strict types, added input validators, and wrote tiny unit tests for a couple of user and chain IDs. That caught any future mismatch immediately.

4. User tower and chain vectors went out of sync.

We once deployed a new user tower while the chain vectors were still the old version, which produced odd rankings.

Fix: We keyed deployments by a single MLflow run ID. The user tower is released only after the batch job lands the matching chain vectors for the same run ID. This removed the race.

5. New restaurants didn't show up (cold start).

Freshly onboarded chains had no history, so they were invisible.

Fix (final design): We built a city-aware, quality-filtered, similarity-based cold-start pipeline:

- Label new vs existing chains per city (recent creation and low impressions).
- Impute popularity fields for new chains using the city's averages from existing chains (so a new Dubai chain looks like an average popular Dubai chain, not a zero).
- Select high-quality local neighbours (existing chains with rating ≥ 4.0 and ≥ 100 ratings).
- Use semantic similarity in our ESE space to keep only close look-alikes (similarity ≥ 0.7 , same city).
- Average their learned embeddings to assign the new chain's initial vector, then overwrite it later as real data arrives.

This made new chains visible and sensibly ranked on day one, without guessing.

6. Edge-case post-processing caused noisy results.

Duplicates from the same brand, empty requests, or “busy” kitchens could pollute the list.

Fix: With the Software Engineer and Vendor team we added simple rules: remove duplicates, filter to available vendors only, and push down “busy” vendors. This kept quality high.

Did it work as expected?

Scope: We shipped what we planned—batch chain vectors, real-time user vectors, and one artifact that reproduces training exactly.

Speed: After the fixes, user-side inference stayed within our ≤ 20 ms target; retrieval plus business filters fit the overall latency budget.

Correctness: Online results matched offline evaluation for our test cases. The early bugs (types, defaults, version skew) are now guarded by code and tests.

Reliability: Using the run ID to sync both towers and keeping one feature module for both training and serving gave us stable, repeatable releases.

Plain takeaway: We had real production problems—too slow, not consistent, wrong data types, version mismatches, and cold-start gaps. We solved them with straightforward steps: preload and cache for speed, one source of truth for features, strict typing and small tests, a single run ID for deployments, and a simple contract for post-processing. The result is a fast, consistent, and easy-to-operate system.

4) Biggest individual learnings

Fix bias at the source

Our first test increased orders but reduced discovery because popular brands dominated. I learned that the right fix is to correct bias during training (how the model learns), not to patch the final list with heuristics. This keeps the system fair, simple, and fast.

Make training and serving identical

Small mismatches between the offline model and the live system (data types, missing-value rules, versions) caused surprising errors. I learned to enforce one shared feature module and one release ID for both training and serving so the model behaves exactly the same everywhere.

Evaluate by slices, not just totals

Overall metrics looked good, but problems appeared in specific “slices” like certain cities, cuisines, and vendor popularity tiers. I learned to set up sliced evaluation from day one so we catch harm early and make trade-offs explicit.

Treat cold start as a must-have

New restaurants were invisible until we added a smart “starter” embedding for them. I learned to design cold-start handling as a core requirement, not an afterthought, so new, high-quality supply is discoverable on day one.

Align with business guardrails up front

Optimizing only for relevance and orders wasn't enough. I learned to agree early on explicit guardrails (discovery, partner share, fee/ETA limits) so success for the model is also success for the business.

Detect and respond to feature drift automatically

We lost time chasing silent failures that came from training on subtly wrong or shifted data. A key learning is to automate drift detection and triage so we quickly decide whether to **fix the data pipeline** (schema/encoding/NULLs) or **fine-tune/retrain** due to a genuine distribution shift. Concretely: lightweight data contracts, per-feature drift monitors (e.g., PSI/KS tests, null/share checks), golden-record online/offline parity, and a retrain-vs-bug decision tree tied to alerts.

If I could redo it

I would ship a standard sliced evaluation dashboard (by city, cuisine, price/rating, popularity, and daypart) in week one and block launches that win overall but lose on critical slices.

I would start with a shared semantic layer and a defined cold-start recipe, so convergence is faster and new vendors appear fairly.

I would lock training/serving parity in code (one feature module, strict typing, a single run ID for both towers) to eliminate silent mismatches.

I would turn business guardrails into executable rules and monitored metrics (for example, "fee/ETA violations per 10k sessions") so we protect marketplace health automatically.

I would add an automated **feature-drift and data-quality gate** (schema checks, per-feature drift thresholds, golden-record parity, and canary retrains) so we catch bugs early and only retrain when the shift is real.

5) LLMs usage

Yes. I used an LLM as an **editorial assistant** to restructure and polish my write-up. I started from my own draft (analysis, metrics, decisions, and outcomes are mine), then used the LLM to improve structure, consistency of headings, clarity, and grammar. It did **not**

generate data, invent results, or perform the analysis/modeling; all numbers and technical content come from my work. I reviewed and approved every suggested change.

Examples of prompts I used:

- “Reformat this text into a clean, professional structure without changing any words or numbers.”
- “Tighten wording for clarity and brevity; keep technical meaning intact and don’t alter metrics.”
- “Turn these notes into a coherent outline with clear section headings; do not add new claims.”
- “Standardize terminology (e.g., ‘orders/session’, ‘Recall@10’) across the document without altering content.”
- “Proofread for grammar and punctuation only—no stylistic rewrites or additions.”

Appendix — How candidate sampling probability debiases the loss

1) Baseline retrieval loss (softmax / InfoNCE)

We score a user-chain pair with:

$$s(u, c) = \frac{\langle \mathbf{u}, \mathbf{r}_c \rangle}{\tau}$$

Per-example loss:

$$\mathcal{L}_{\text{base}} = -\log \frac{e^{s(u, c^+)}}{e^{s(u, c^+)} + \sum_j e^{s(u, c_j^-)}}$$

If negatives come from logs, popular chains appear more often; they dominate the denominator and the gradients.

2) Exposure-aware correction with candidate sampling probabilities

Let $q(c)$ be the candidate sampling probability (our exposure proxy) for chain c . Two equivalent ways to correct:

a) Logit correction (subtract $\log q$):

$$\tilde{s}(u, c) = s(u, c) - \log q(c)$$

Use the corrected logit in the same softmax loss. Head chains (large q) are down-weighted; tail chains (small q) are up-weighted.

b) Importance weighting (inverse-propensity flavor):

$$\mathcal{L}_{\text{ips}} = -\log \frac{e^{s(u, c^+)}}{e^{s(u, c^+)} + \sum_j \frac{1}{q(c_j^-)} e^{s(u, c_j^-)}}$$

In our implementation we pass `candidate_sampling_probability` and `candidate_ids` into the TFRS task (plus `sample_weight` on positives), which is equivalent to the corrections above.

3) What it does to gradients (intuition)

Softmax gradients w.r.t. a negative's logit are roughly its probability:

$$\frac{\partial \mathcal{L}}{\partial s(u, c^-)} \approx p(c^- | u)$$

After correction with (a) the effective probability becomes:

$$\tilde{p}(c^- | u) \propto \frac{e^{s(u, c^-)}}{q(c^-)}$$

So head negatives (large q) get less gradient; tail negatives (small q) get more gradient. Training stops collapsing onto the head.

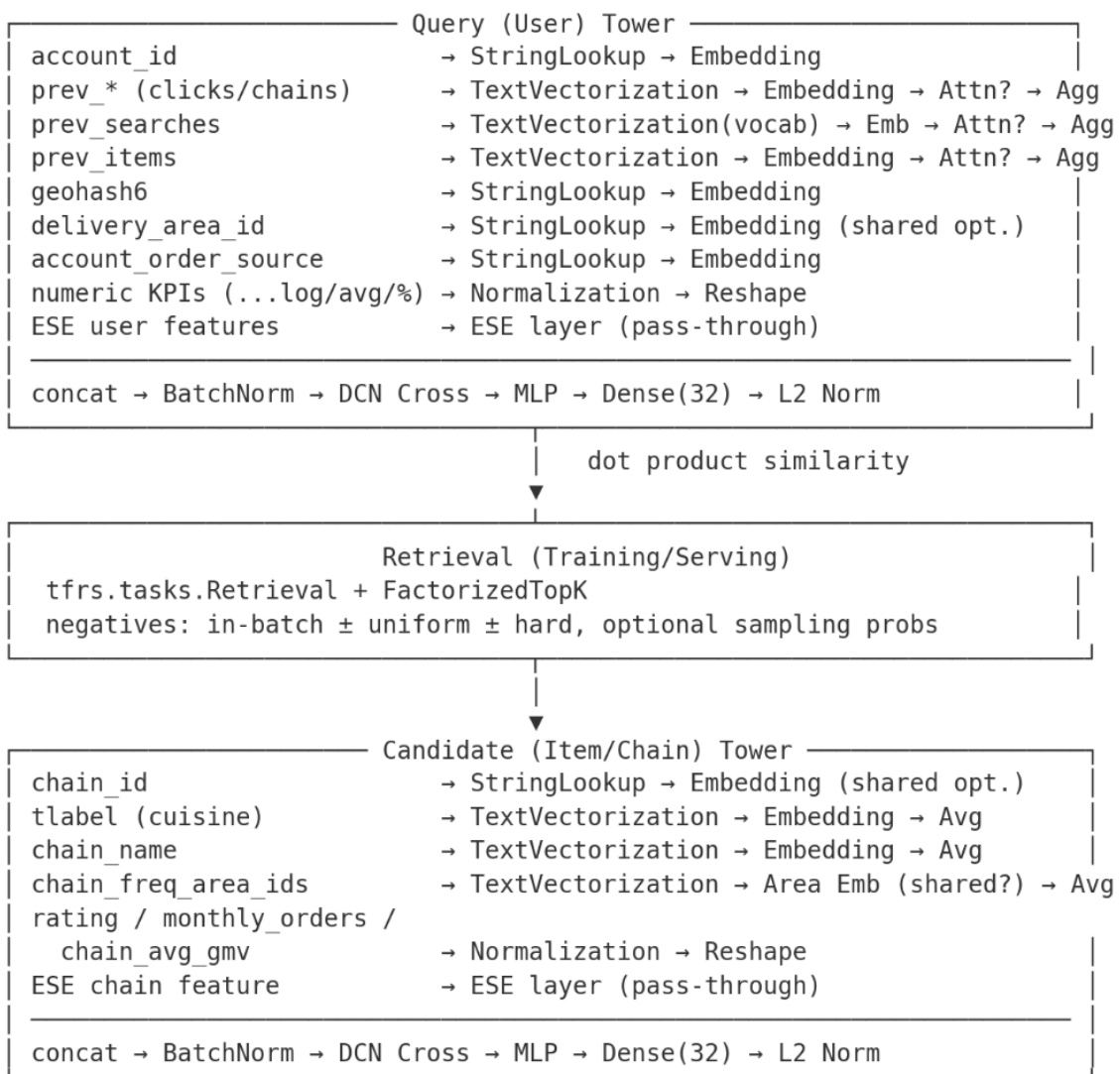
4) Positive weights and temperature

- We also apply sample weights on positives so frequent head positives don't dominate.
- A small temperature τ prevents early over-confidence (over-peaky softmax) and stabilizes learning.

5) Tiny numeric intuition

If Chain A appears in 40% of training batches and Chain B in 2%, then without correction A gets $\sim 20\times$ more gradient updates. With exposure-aware correction (subtract $\log q$ or weight by $1/q$), A's influence is scaled down and B's is scaled up so both compete on true relevance, not raw exposure. **Takeaway:** Adding `candidate_sampling_probability` is like subtracting $\log q(c)$ from each candidate's logit (or weighting by $1/q(c)$). This rebalances the softmax so the model learns relevance rather than how often a chain appeared in the logs.

Appendix: Two towers Architecture



Example:

Query (User) Tower

account_id	→ [0.20, 0.10, -0.10]
prev_chains avg [c2,c3,c2]	→ [0.43, 0.13, 0.23]
prev_searches ["sushi","ramen"]	→ [0.45, 0.15, 0.05]
delivery_area_id=A12	→ [0.00, 0.30, 0.10]
geohash6=ulabcd	→ [0.10, -0.20, 0.00]
order_hour=19	→ [0.05, 0.10, -0.05]

concat → head → user vector $u = [1.23, 0.58, 0.23]$

dot product similarity

Retrieval (Training/Serving)

score(u , SushiGo) = $u \cdot v_{\text{pos}} = 2.00$
score(u , BurgerBarn) = $u \cdot v_{\text{neg}} = 1.19$
→ SushiGo ranks higher ($2.00 > 1.19$)

Candidate (Item/Chain) Tower

SushiGo:

chain_id=c2	→ part of v_{pos}	
tlabel='japanese sushi'	→ part of v_{pos}	
chain_name='Sushi Go'	→ part of v_{pos}	$v_{\text{pos}} = [1.25, 0.60, 0.45]$
freq_area_ids=A12	→ part of v_{pos}	

BurgerBarn:

chain_id=c1	→ part of v_{neg}	
tlabel='burger'	→ part of v_{neg}	
chain_name='Burger Barn'	→ part of v_{neg}	$v_{\text{neg}} = [0.35, 1.20, 0.25]$
freq_area_ids=A99	→ part of v_{neg}	